

Apuntes_NumPy_3_Otros

May 11, 2026

1 Apuntes NumPy 3: Otros

Una vez visto los arrays y como operar con ellos, vamos a dedicar este archivo a otros elementos que estan incluidos en esta biblioteca que son interesantes conocer.

Entre las cosas que vamos a ver veremos:

- *Datetime* y como operar con ellos
- *Broadcasting* extendido
- ¿Como realizar copias de los trabajos?
- Lectura de ficheros csv : *np.genfromtext()*

1.1 *Datetime64*

Es un tipo de dato que ofrece la libreria NumPy, que sirve para poder trabajar con fechas.

En el analisis de datos se trabaja principalmente con *Numpy* como motor de calculo y *Pandas* como libreria de gestion de datos macro, y dentro de ese maremagnum de información, las fechas es un tipo de dato que se puede trabajar de manera independiente y mas eficiente.

El dato *datetime* ya existe en Python base, pero ***datetime64*** es el que incluye NumPy que amplia su funcionalidad.

Es un tipo de dato que se utiliza con **funciones de Numpy**, tal y como es el caso de *np.arange()*

Usar esta funcion nos permitira indicarle que nos genere un array con fechas en un rango en base a si queremos que sean *días, meses o años*

np.arange ("fecha start" , "fecha stop" , dtype = "datetime64[“Día, Mes, Año”])

Los *dtype* seran ***[D]*** para días, ***[M]*** para meses, ***[Y]*** para años

El formato de escritura de fecha sera el **britanico**: Año - Mes - Día

Tambien se pueden realizar **operaciones matematicas** con ellos, como sumar, restar...

Usar esta funcion nos permitira operar los datos de fecha como si fueran int o float, pero con las reglas de fechas

np.datetime64("fecha") "+ (u otro operador)" np.timedelta64 ("numero" , ["Día, Mes, Año"])

Los *dtype* serán *[D]* para días, *[M]* para meses, *[Y]* para años
np.timedelta64("numero", "tipo de dato fecha") permite cambiar los tipos de datos de las fechas para poder operar con ellos.

```
[6]: import numpy as np

fecha = np.arange("2023-04-12" , "2025-02-04", dtype = "datetime64[Y]")
print(fecha)

['2023' '2024']
```

TIP: Tienes que poner el tipo de dato *sin espacios* junto a *datetime64* sino dara error

```
[4]: import numpy as np

fecha = np.arange("2023-04-12" , "2025-02-04", dtype = "datetime64 [D]")
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[4], line 3
      1 import numpy as np
----> 3 fecha = np.arange("2023-04-12" , "2025-02-04", dtype = "datetime64 [D]"

TypeError: Invalid datetime metadata string " [D]" at position 1
```

```
[10]: import numpy as np

fecha_inicial = np.datetime64("2025-12-10") - np.timedelta64 (75,"D")
print(fecha_inicial)

2025-09-26
```

1.2 *Broadcasting* (extendido)

Es el proceso que realiza la libreria NumPy al tratar arrays con forma distinta cuando realiza operaciones matematicas con ellos

Es un proceso automatico de ajuste entre el array mas pequeño y el mas grande, que convierte al array pequeño en una *version ampliada del mismo* para poder operar con el grande.

Este proceso es esencial para el funcionamiento de Numpy ya que:

- Hace que sea mas eficiente, no consume memoria extra al realizar las dimensionalidades
- Combina operaciones matematicas para optimizar la capacidad de computacion y uso de memoria

Para poder utilizarlo a nuestro favor, existen algunas reglas para operar con arrays de 2 dimensiones (sino dara un error):

- Deben tener la misma **la misma forma**
- Una de ellas es **1 dimension de 1 elemento**

Esto puede ser un *int* , un *float*

Puede ser un array de 1 dimension

Puede ser un array de 2 dimensiones x filas y 1 columna

Es posible a modo de excepcionalidad, *forzar* a Python a que pueda realizar el calculo entre los 2 array que le hemos lanzado, esto se realiza **añadiendo ejes nuevos** (nuevas dimensiones) al array menor para poder *completar la forma adecuada*.

“nombre array” = “nombre array pequeño”[:, np.newaxis]

En definitiva lo que permite es generar arrays de formas (x, 1) para que ya se puedan operar

```
[21]: import numpy as np

array_grande = np.array([2,5,7,2,6])
array_pequeño = np.array([4,6,7,4])
a = array_grande
b = array_pequeño
print(a+b)
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[21], line 7
      5 a = array_grande
      6 b = array_pequeño
----> 7 print(a+b)

ValueError: operands could not be broadcast together with shapes (5,) (4,)
```

```
[23]: import numpy as np

array_grande = np.array([2,5,7,2,6])
array_pequeño = np.array([4,6,7,4])
a = array_grande
b = array_pequeño
b_plus = b[:, np.newaxis]
print(b_plus)
print( 40*"-")
print(b_plus+a)
```

```
[[4]
 [6]
 [7]
 [4]]
```

```
-----  
[[ 6  9 11  6 10]  
 [ 8 11 13  8 12]  
 [ 9 12 14  9 13]  
 [ 6  9 11  6 10]]
```

1.3 ¿Como realizar copias de Arrays?

Hemos visto que los arrays permiten realizar muchas modificaciones de datos, que generalmente guarda en una nueva copia de datos.

Pero en otros casos, **NumPy modifica la variable original** por procesos de eficiencia (por ejemplo en el *boadcasting*)

De modo que tenemos en tener en cuenta cuando sucede esto y actuar en consecuencia para no modificar el array original.

Existen 3 casuisticas que vamos a conocer.

1.3.1 Trabajar sin copia de datos: *referencia*

Este es la versión **de error mas común** y sucede cuando asignamos un array a otra variable.

Lo que sucede es que se **define una nueva referencia en memoria** pero no sea crea una copia de datos, por lo que **modificara al array original**

“nombre array copia” = “nombre array original”

Cualquier cambio en *“la copia”* cambiara al original

Es lo mismo que con otras estructuras de datos, si le asignas un valor a una variable directamente de otra, *las vinculas*

```
[26]: import numpy as np  
  
array_original = np.arange(1,5)  
print(array_original)  
print(40*"-")  
array_copia = array_original  
array_copia[2] = 300  
print(array_original)
```

```
[1 2 3 4]
```

```
-----  
[ 1  2 300  4]
```

1.3.2 Trabajar copia superficial de datos: *Vista*

Este es la versión que **sucede cuando modificamos o dividimos un array en otro(s)**

Es decir, al utilizar, *.reshape()* o *.split()* o cualquiera de sus variantes

Podremos modificar la forma del array original, **pero si cambiamos un elemento en el array modificado tambien cambiara en el original**

Solo los cambios de los valores de los elementos en “la copia” cambiara al original

```
[33]: import numpy as np

array_original = np.arange(1,13)
print(array_original)
print(40*"-")
array_copia = array_original.reshape(3,4)
print(array_copia)
print(40*"-")
array_copia[1, 3] = 300
print(array_copia)
print(40*"-")
print(array_original)
```

```
[ 1  2  3  4  5  6  7  8  9 10 11 12]
-----
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
-----
[[ 1  2  3  4]
 [ 5  6  7 300]
 [ 9 10 11 12]]
-----
[ 1  2  3  4  5  6  7 300  9 10 11 12]
```

1.3.3 Trabajar copia profunda de datos: *copy()*

El modo correcto de trabajar copias de arrays sin correr riesgos a modificaciones en el original es utilizando el metodo *copy()*

“nombre array copia” = “nombre array original”.*copy()*

Generamos una copia exacta de forma y contenido sin compartir estructura ni valores en memoria del array original

```
[34]: import numpy as np

array_original = np.arange(1,5)
print(array_original)
print(40*"-")
array_copia = array_original.copy()
array_copia[2] = 300
print(array_original)
```

```
[1 2 3 4]
-----
[1 2 3 4]
```

1.4 Lectura de ficheros csv : *np.genfromtxt()*

Por ultimo, hemos visto como la libreria Numpy nos permite trabajar con datos a traves de los array y muchas posibilidades, pero nos falta un aspecto fundamental para trabajar con datos, (que exploraremos mas en la siguiente unidad con la libreria *pandas*)

¿Como leemos ficheros con datos externos?¿Como accedemos a ellos?

Sin meternos en mucha harina, indicaremos ahora como podemos a traves de esta libreria **trabajar con archivos csv** de manera sencilla.

Es habitual utilizar ficheros de tipo CSV como formato de intercambio de datos, ya que es un formato en texto plano sencillo de manipular y que está soportado por la mayoría de software de manejo de datos, tanto para importar como para exportar información.

El metodo *np.genfromtxt()* permite leer ficheros csv e importar sus datos al entorno.

“nombre array” = np.genfromtxt(fname = “ruta de acceso al fichero terminado en .csv”,

delimiter = “; (u otro signo delimitador de datos)”

, skip_header = 0)

Para amplair informacion(<https://numpy.org/doc/stable/user/basics.io.html>)